

# База знань DRAKON: Архітектура, семантика та інженерний контекст

## 1. Executive Summary

Візуальна алгоритмічна мова DRAKON (абр. «Дружелюбный Русский Алгоритмический язык, Который Обеспечивает Наглядность») становить собою унікальний підхід до програмування та моделювання бізнес-процесів, що інтегрує строгу математичну логіку з принципами когнітивної ергономіки. Розроблена в межах радянської космічної програми «Буран», мова була створена для подолання бар'єрів розуміння між фахівцями різних дисциплін шляхом заміни складного текстового коду на уніфіковані, інтуїтивно зрозумілі візуальні структури.<sup>1</sup> В основі архітектури DRAKON лежать топологічні інваріанти: повна заборона на перетин ліній зв'язку, строге використання ортогональної (Манхеттенської) геометрії, а також концепція єдиної вертикальної осі («шампура»), що визначає «найщасливіший шлях» (happy path) алгоритму.<sup>3</sup>

На відміну від традиційних специфікацій ISO 5807 або UML, DRAKON не просто візуалізує логіку, але й діє як гібридна мова (DRAKON-2), де графічні макроікони містять фрагменти реального коду цільових мов (C, Java, Erlang, JavaScript тощо) для прямої трансляції в абстрактне синтаксичне дерево (AST) компілятора.<sup>2</sup> Завдяки наявності серіалізованих машиночитаних форматів, таких як JSON та SQLite, топологічна структура діаграм відділена від геометричного рендерингу. Це робить DRAKON надзвичайно перспективним стандартом для інтеграції з системами штучного інтелекту, де великі мовні моделі (LLM) здатні генерувати логічні графи бізнес-процесів, а детерміновані рушії автоматично розраховують їхнє ідеальне ергономічне розміщення.<sup>5</sup>

## 2. Glossary

Для забезпечення чіткості подальшого аналізу необхідно визначити ключові концептуальні терміни, що формують архітектурний базис мови DRAKON. Ці терміни визначають як семантичну структуру алгоритмів, так і візуальні парадигми їхнього відображення.

- **DRAKON (ДРАКОН):** Базова візуальна мова моделювання, алгоритміки та програмування. Її назва розшифровується як мова, що забезпечує наочність (ясність). Функціонує як інструмент посилення інтелекту (intelligence augmentation), оскільки її правила когнітивно оптимізовані для легкого сприйняття людським мозком.<sup>1</sup>
- **Primitive (Примітив):** Найпростіший структурний тип діаграми в системі DRAKON, призначений для моделювання лінійних або базових розгалужених

алгоритмів. Примітив завжди будується навколо єдиної головної вертикальної осі та має лише одну точку входу і одну точку виходу.<sup>7</sup>

- **Silhouette (Силует):** Складний макро-шаблон діаграми, який розв'язує проблему масштабування шляхом просторової декомпозиції. Силует розрізає великий алгоритм на логічні частини («гілки»), що розташовуються паралельно по горизонталі, дозволяючи читачу одночасно сприймати як макроструктуру (заголовки гілок), так і мікроструктуру (деталі всередині гілок) алгоритму.<sup>7</sup>
- **Skewer (Шампур):** Центральна вертикальна лінія гілки або примітиву. Вона виконує роль топологічного хребта алгоритму, на який «нанизуються» графічні ікони. Шампур завжди відображає найбільш бажаний та найчастіше використовуваний шлях виконання програми («happy path»)<sup>3</sup>
- **Icon (Ікона):** Базовий неподільний графічний елемент («літера» візуального алфавіту), що має конкретне алгоритмічне призначення, наприклад, ікона перевірки умови або ікона дії.<sup>2</sup>
- **Macroicon (Макроікона):** Складена графічна конструкція («слово»), утворена з кількох базових ікон, поєднаних за суворими правилами. Прикладами є цикли For, структури Select-Case або паралельні процеси.<sup>2</sup>
- **Valence Points (Валентні точки):** Специфічні точки з'єднання всередині макроікон, в які можуть бути послідовно вставлені інші ікони або мікроікони, формуючи колонки дій.<sup>2</sup>
- **ERIL (Entity-Relationship and Inheritance Language):** Споріднена з DRAGON візуальна мова, розроблена Степаном Міткіним, яка застосовує когнітивні правила DRAGON (ортогональні лінії, відсутність перетинів, просторове розділення ієрархій) для моделювання структур даних, реляційних зв'язків та об'єктно-орієнтованого успадкування.<sup>4</sup>
- **DRAGON-2 (Гібридна мова):** Механізм використання візуального каркаса DRAGON у комбінації з текстовим синтаксисом існуючих мов програмування (наприклад, DRAGON-JavaScript, DRAGON-C). Ікони визначають топологію та потік управління (flow control), а текст всередині ікон визначає низькорівневі інструкції.<sup>2</sup>

### 3. Core Principles of DRAGON

#### Походження та контекст створення

Історія виникнення мови DRAGON глибоко вкорінена в кризові явища розробки програмного забезпечення для складних кібернетичних систем у 1980-х роках. Мова була розроблена в період з 1986 по 1996 рік у Радянському Союзі під керівництвом В.Д. Паронджанова як фундаментальний компонент космічної програми «Буран».<sup>1</sup> Під час розробки штучного інтелекту та систем управління космічним кораблем інженери стикнулися з проблемою експоненційного

зростання складності комунікації. Різні підрозділи використовували несумісні вузькоспеціалізовані мови програмування: PROL2 (для бортових обчислювальних систем), DIPOL (для наземних систем обслуговування) та LAKS (для моделювання).<sup>2</sup>

Головним усвідомленням розробників «Бурану» стало те, що пояснення логіки алгоритму людині є значно складнішою задачею, ніж безпосереднє проєктування чи машинна імплементація цього алгоритму.<sup>3</sup> Традиційний текстовий код та тогочасні блок-схеми створювали величезне когнітивне навантаження, призводячи до помилок, які виявлялися лише на стадії тестування. Відповіддю на цю проблему стала розробка єдиної, універсальної візуальної мови, яка б замінила розрізнені інструменти та забезпечила абсолютну прозорість логіки для будь-якого інженера.<sup>2</sup> Згодом, в епоху "гласності", мова DRAGON була розсекречена, і завдяки зусиллям ентузіастів почала адаптуватися не лише в аерокосмічній сфері, але й у хімії, юриспруденції, медицині та бізнес-аналітиці.<sup>1</sup>

## Принципи читабельності та Human-Centered Design

Фундаментальна філософія DRAGON формулюється девізом: **«Clarity is above all»** (Ясність понад усе) або «Глянув — і зрозумів».<sup>3</sup> Цей принцип реалізується шляхом поєднання строгої математичної бази (здатності формалізувати будь-який обчислювальний процес) з когнітивною ергономікою. Ергономіка в контексті DRAGON визначається як мистецтво перетворення виснажливої та нудної роботи з аналізу коду на легкий та комфортний процес.<sup>3</sup>

Дослідження сприйняття візуальної інформації лягли в основу ключових топологічних парадигм мови. По-перше, DRAGON використовує гравітаційну метафору читання. У фізичному світі об'єкти рухаються вниз під дією гравітації, тому людські очі сприймають рух зверху вниз як найбільш природний та такий, що не потребує додаткових зусиль.<sup>3</sup> Відповідно, часовий та логічний потік алгоритму в DRAGON завжди спрямований донизу.

По-друге, просторова організація елементів чітко корелює з їхньою логічною успішністю. Правило «Головного маршруту» вимагає, щоб найоптимальніший сценарій виконання програми («happy path») лежав виключно на прямій вертикальній лінії.<sup>3</sup> Око читача не повинно здійснювати горизонтальних пошукових рухів для розуміння основної мети алгоритму. Будь-яке відхилення від ідеального сценарію маршрутизується праворуч. У мові діє жорстке правило вторинних маршрутів: «Чим далі вправо — тим гірше».<sup>3</sup> Це означає, що крайня права частина діаграми автоматично сприймається мозком як зона обробки фатальних помилок або виняткових ситуацій.

## Відмінності від звичайних Flowchart-нотацій

На противагу традиційним блок-схемам, описаним у стандарті ISO 5807-85 або

специфікаціях UML, які часто перетворюються на заплутане візуальне павутиння («cobweb»), DRAKON накладає серію суворих топологічних заборон:

1. **Геометрія без перетинів:** У DRAKON повністю заборонені будь-які перетини ліній зв'язку.<sup>4</sup> Людський мозок підсвідомо генерує хибне припущення про наявність логічного вузла там, де графічні лінії перетинаються, вимагаючи додаткових когнітивних зусиль для спростування цього факту. DRAKON усуває цю проблему на структурному рівні.
2. **Манхеттенська маршрутизація:** Використання діагональних, хвилястих або вигнутих ліній суворо заборонено.<sup>4</sup> Око витрачає мікрозусилля на відстеження складних кривих. Застосування виключно ортогональних ліній (вертикальних та горизонтальних) під прямим кутом знімає напругу із зорового апарату та обмежує кількість можливих сусідніх вузлів до чотирьох, роблячи граф передбачуваним.<sup>13</sup>
3. **Ієрархія без приховування:** Традиційний підхід боротьби зі складністю («divide and conquer») полягає в приховуванні деталей у вкладених функціях або саб-рутинах. DRAKON, завдяки макро-шаблону Силует, дозволяє виводити два рівні ієрархії на одне полотно, запобігаючи ефекту «чорного ящика» та полегшуючи наскрізний аудит алгоритму.<sup>3</sup>

## Сучасне практичне використання

Сьогодні DRAKON використовується в широкому спектрі інженерних та нетехнічних дисциплін. Німецький аерокосмічний центр (DLR) використовує DRAKON-графіки для генерації С-коду, що імплементує критичні функції під час льотних випробувань (наприклад, детектування запуску ракет).<sup>2</sup> Розробники застосовують його для створення складних кінцевих автоматів (state machines), оскільки DRAKON унікально поєднує дерева прийняття рішень з картами станів.<sup>4</sup> Крім того, мова набула популярності в моделюванні бізнес-процесів та створенні медичних алгоритмічних інструкцій для персоналу, де ціною помилки сприйняття є людське життя.<sup>2</sup>

## 4. DRAKON Icons Catalog

 **Примітка щодо реконструкції:** Згідно з офіційною документацією та академічними статтями, повний графічний алфавіт мови DRAKON складається з 27 базових ікон та 21 макроікони.<sup>2</sup> Проте, відкриті джерела містять детальний опис лише ключової підмножини елементів. Наведена нижче база даних є максимально повною реконструкцією на основі зібраних технічних специфікацій.<sup>14</sup>

Кожна ікона в системі DRAKON має жорстко задане алгоритмічне призначення та правила розміщення у топологічному просторі діаграми.

1. **Стартові та термінальні вузли:** Будь-який алгоритм вимагає чітко визначених

меж. Ікона Start/Begin (заголовок) використовується виключно у верхній частині найлівішого шампура або як вхідна точка гілки. Вона відповідає на запитання «Як називається ця процедура?». Ікона End термінує виконання і повинна бути унікальною для всієї діаграми (розташовується внизу крайнього правого шампура), забезпечуючи чітку структуру «від початку до кінця».<sup>3</sup>

**2. Дії та обчислення:** Ікона Action є прямокутником, що містить імперативні команди, формули або виклики функцій.<sup>14</sup> Її розташування обмежується лише вертикальними лініями шампура, і вона виконується лінійно зверху вниз.

**3. Управління потоком (Розгалуження):** Ікона Question (Problem/If) утворює біфуркацію. Важливе обмеження: виходи можливі лише вниз (зазвичай Yes) та вправо (зазвичай No).<sup>3</sup> Зміна етикеток допускається, проте лівобічні виходи категорично заборонені, щоб не руйнувати гравітаційне читання. Для множинного вибору використовується зв'язка ікон Choice (запитання) та горизонтального ряду Case (варіанти відповідей), де порожній Case праворуч діє як блок default.<sup>14</sup>

**4. Інтеграція підпроцесів:** Ікона Insertion дозволяє викликати іншу DRAKON-схему. Поточний потік виконання зупиняється і чекає на повернення результату з підпрограми.<sup>14</sup> Це фундаментальний інструмент перевикористання коду.

**5. Циклічні операції:** Цикли формалізуються за допомогою макроікон. For loop візуально відображається двома іконами (початок і кінець циклу), між якими розміщується тіло алгоритму. Він застосовується як для ітерації колекцій, так і для лічильників.<sup>14</sup>

**6. Ввід/Вивід та взаємодія із зовнішнім світом:** Ікони Input та Output мають специфічну двокамерну структуру. Верхня частина містить дієслово (наприклад, "отримати", "відправити"), а нижня — іменник або опис об'єкта.<sup>14</sup> Це стимулює інженера писати чітко структуровані команди. Існують також спрощені однокамерні версії Simple Input/Output.<sup>14</sup>

**7. Управління ресурсами та часом:** DRAKON містить потужні інструменти для програмування систем реального часу. Ікона Timer ініціалізує лічильник часу. Відповідна їй ікона Duration, яка приєднується до інших ікон з лівого боку, затримує виконання наступної дії на заданий час (відносно запуску таймера). Макроікона Control period (що складається зі Start та End) окреслює межі критичного блоку коду, виконання якого має завершитись за чітко визначений інтервал (deadlines).<sup>14</sup>

**8. Паралелізм та асинхронність:** Макроікона Concurrent processes формує дві або більше паралельні гілки потоків.<sup>14</sup> Ікона Process дозволяє запустити асинхронну фонову задачу (з командами Start, Pause, Continue, Stop) без блокування поточного алгоритму. Зв'язок із таким процесом підтримується через ікони вводу/виводу.<sup>14</sup>

**9. Багатоцільова ікона «Полиця» (Shelf):** Дана ікона адаптує свою семантику залежно від контенту двох її секцій.<sup>14</sup>

- *Наказ актору:* Верхня полиця — актор (наприклад, Сервер), Нижня — команда (наприклад, Очистити кеш).
- *Повідомлення:* Верхня — відправник і отримувач, Нижня — зміст повідомлення (payload).
- *Дія:* Верхня — фраза дії, Нижня — об'єкт маніпуляції.
- *Присвоєння:* Верхня — ім'я змінної, Нижня — присвоєне значення.<sup>14</sup>

| Icon                 | Альтернативні назви | Призначення                                 | Допустима позиція                        | Обмеження                                    | Типові помилки                                                 |
|----------------------|---------------------|---------------------------------------------|------------------------------------------|----------------------------------------------|----------------------------------------------------------------|
| <b>Start / Begin</b> | Header              | Точка входу алгоритму. <sup>1</sup>         | Зверху найлівішого шампура. <sup>3</sup> | Лише одна на схему/гілку.                    | Розміщення кількох точок входу.                                |
| <b>End</b>           | Кінець              | Фізичне і логічне завершення. <sup>1</sup>  | Внизу правого шампура. <sup>8</sup>      | Строго одна на весь алгоритм. <sup>3</sup>   | Використання кількох End для "early return". <sup>3</sup>      |
| <b>Action</b>        | Process, Дія        | Виконання команд. <sup>1</sup>              | На осі шампура.                          | Відсутні.                                    | Приховування умовної логіки в тексті ікони.                    |
| <b>Question</b>      | Problem, If         | Логічне розгалуження (умова). <sup>14</sup> | На осі шампура.                          | Виходи виключно вниз і вправо. <sup>14</sup> | Лівобічні виходи, що ламають гравітаційний потік. <sup>3</sup> |
| <b>Choice</b>        | Selection, Switch   | Точка розгалуження множинног                | На осі шампура.                          | Мусить з'єднуватись із серією                | Вставлення дій між Choice та                                   |

|                     |                |                                                       |                                  |                                                                     |                                                                |
|---------------------|----------------|-------------------------------------------------------|----------------------------------|---------------------------------------------------------------------|----------------------------------------------------------------|
|                     |                | о вибору. <sup>14</sup>                               |                                  | ікон Case. <sup>3</sup>                                             | Case.                                                          |
| <b>Case</b>         | Option         | Варіант вибору або default (порожній). <sup>14</sup>  | Рядом по горизонталі під Choice. | Працює лише в макроіконі Choice.                                    | Відрив ікони від батьківського блоку.                          |
| <b>Insertion</b>    | Call, Виклик   | Синхронний виклик підпрограми. <sup>1</sup>           | На осі шампура.                  | Виконання зупиняється до завершення підпрограми.                    | Плутанина з асинхронним Process.                               |
| <b>For loop</b>     | Цикл           | Ітерація по колекції або діапазону. <sup>14</sup>     | На осі шампура.                  | Потребує Begin та End ікон циклу. <sup>3</sup>                      | Хаотичне зв'язування стрілок для створення циклу. <sup>3</sup> |
| <b>Input/Output</b> | Ввід / Вивід   | Обмін даними.                                         | На осі шампура.                  | Жорстке розділення на дві секції: дієслово + іменник. <sup>14</sup> | Запис неструктурованого тексту в одну секцію.                  |
| <b>Shelf</b>        | Полиця         | Наказ актору, повідомлення, присвоєння. <sup>14</sup> | На осі шампура.                  | Повинна відповідати одному з чотирьох патернів. <sup>14</sup>       | Використання замість ікони Action без причини.                 |
| <b>Process</b>      | Фоновий процес | Асинхронний запуск паралельної процедури.             | На осі шампура.                  | Алгоритм процесу має існувати в іншій                               | Очікування результату (блокування потоку).                     |

|                 |            |                                                              |                                                           |                                                                        |                                                                 |
|-----------------|------------|--------------------------------------------------------------|-----------------------------------------------------------|------------------------------------------------------------------------|-----------------------------------------------------------------|
|                 |            | 14                                                           |                                                           | діаграмі. <sup>14</sup>                                                |                                                                 |
| <b>Duration</b> | Тривалість | Таймінг або затримка (залежить від контексту). <sup>14</sup> | Прикріплен а <b>ліворуч</b> до іншої ікони. <sup>14</sup> | Потребує ікони Timer вище по потоку для роботи як delay. <sup>14</sup> | Приєднання ікони з правого боку.                                |
| <b>Address</b>  | Адреса     | Навігація до наступної гілки в Силуеті. <sup>1</sup>         | В кінці гілки.                                            | Вказує лише на ім'я іншої гілки (в її початок). <sup>8</sup>           | Стрибок усередину іншої гілки (Spaghetti pattern). <sup>3</sup> |
| <b>Comment</b>  | Коментар   | Текстове пояснення коду. <sup>14</sup>                       | Прикріплюється горизонтально до елемента.                 | Ігнорується компілятором.                                              | Запис алгоритмічної логіки в коментар.                          |

## 5. Rules of Diagram Construction

Правила побудови в системі DRAGON мають глибоке концептуальне обґрунтування, яке можна порівняти із теорією задоволення обмежень (Constraint Satisfaction Problems, CSP). У цій теорії обмеження поділяються на *Жорсткі* (Hard constraints), що формують множину допустимих розв'язків (їх порушення робить систему невалідною), та *М'які* (Soft constraints), що оптимізують якість розв'язку.<sup>15</sup> DRAGON використовує подібну дихотомію для регламентації малювання алгоритмів.

### Hard Constraints (Топологічні інваріанти)

Жорсткі правила забезпечують математичну однозначність та компільованість графа. До них належать:

- **Правило ортогональності та перетинів:** Геометрія ліній має бути строго "манхеттенською" (лише вертикальні та горизонтальні сегменти). Перетини ліній категорично заборонені.<sup>4</sup> Це правило усуває необхідність "слідкувати за дротом", оскільки топологія графа стає планарною.
- **Правило одного терміналу:** Будь-який алгоритм (навіть такий, що має складну логіку раннього виходу або обробки помилок) має завершуватись єдиною іконою End.<sup>3</sup> Переходи до завершення алгоритму моделюються маршрутизацією ліній до цього єдиного вузла, забезпечуючи чітку парадигму

«перед-умова / після-умова».

- **Векторність ікон розгалуження:** Ікони If/Question зобов'язані мати виходи виключно вниз і вправо.<sup>3</sup> Заборона лівобічних виходів запобігає перетворенню діаграми на хаотичне скупчення зворотних петель і захищає цілісність головного шампура.
- **Заборона стрибків усередину абстракцій:** Використання ікон навігації (Address) для стрибка безпосередньо в середину іншої алгоритмічної гілки розцінюється як фатальна помилка, що руйнує модульність («чорний ящик» гілки).<sup>3</sup>

## Soft Constraints (Ергономічні евристики)

М'які правила спрямовані на зменшення когнітивного навантаження під час візуального аудиту та зневадження коду.

- **Правило центрального шампура (Happy Path):** В ідеально побудованій діаграмі головний маршрут — сценарій, що веде до найбільшого успіху і найменших витрат ресурсів — повинен бути розташований на головній вертикальній осі.<sup>3</sup> Це дозволяє архітектору миттєво зрозуміти базову мету алгоритму без читання умовних гілок. Зсув happy path убік вважається порушенням ергономіки, що призводить до "поламаного шампура".<sup>3</sup>
- **Евристика погіршення («Чим правіше, тим гірше»):** Альтернативні маршрути ранжуються по осі X. Найгірші сценарії (наприклад, критичні збої бази даних, обробка винятків) повинні знаходитись на крайніх правих вертикалях.<sup>3</sup> Це правило дозволяє експерту швидко локалізувати блоки обробки помилок.
- **Організація Силуетів:** У складних структурах гілки повинні бути відсортовані зліва направо у хронологічному порядку їх виконання, щоб просторова візуалізація відображала розгортання процесу в часі.<sup>8</sup>

| Правило                             | Тип (Hard/Soft) | Що порушується при недотриманні                                              | Можливість авто-перевірки                                            |
|-------------------------------------|-----------------|------------------------------------------------------------------------------|----------------------------------------------------------------------|
| Відсутність перетинів <sup>13</sup> | Hard            | Топологічна валідність. Алгоритм стає неоднозначним та візуально агресивним. | Рушії (DRAKON Editor) гарантують відсутність перетинів алгоритмічно. |

|                                                  |      |                                                                                          |                                                                                                   |
|--------------------------------------------------|------|------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| Один End на алгоритм <sup>3</sup>                | Hard | Компілятор не зможе коректно згенерувати закриваючий scope (наприклад, return у C++).    | Перевірка абстрактного синтаксичного дерева (AST) на кількість термінальних вузлів.               |
| Виходи з If вниз/вправо <sup>3</sup>             | Hard | Руйнується патерн візуальної логіки. Виникає ефект заплутаності ліній.                   | Автоматично блокується на рівні UI редакторів та JSON-валідаторів.                                |
| Happy Path на шампурі <sup>3</sup>               | Soft | Втрата ефекту "глянув-і-зрозумів". Читач витрачає час на пошук головної логіки.          | Складна авто-перевірка. Потребує семантичного аналізу коду за допомогою AI або ручного код-рев'ю. |
| Чим правіше, тим гірше <sup>3</sup>              | Soft | Когнітивний дисонанс: позитивні рішення знаходяться там, де око очікує побачити помилки. | Потребує AI Critic моделі для визначення семантичної "успішності" блоку.                          |
| Стрибки в середину гілки заборонені <sup>3</sup> | Hard | Еквівалент шкідливого goto. Руйнується концепція скінченного автомата.                   | Топологічний валідатор здатний перевірити, чи посилання вказує лише на стартові вузли гілок.      |

## 6. Silhouette

Для розв'язання проблеми експоненційного зростання складності програмного забезпечення розробники DRAKON впровадили унікальну макроструктуру — **Силует (Silhouette)**. У той час як базова діаграма («Примітив») призначена для моделювання лінійних потоків і відображається як єдина нерозривна сутність,

Силует діє як архітектурний інструмент розділення та володарювання (divide and conquer).<sup>3</sup>

## Архітектура Силуету

Силует розрізає монолітний алгоритм на семантично незалежні блоки, які називаються «гілками» (branches). Ці гілки вишиковуються паралельно одна одній на одному візуальному полотні. Силует завжди надає відповіді на «Три питання короля», які дозволяють читачу миттєво зрозуміти суть алгоритму<sup>3</sup>:

1. **Яка мета цього алгоритму?** Відповідь міститься в іконі-заголовку у лівому верхньому кутку.
2. **Скільки логічних кроків містить алгоритм?** Відповідь дає візуальна кількість паралельних гілок.
3. **Яка суть кожного кроку?** Відповідь забезпечується іконами-заголовками, що вінчають кожну гілку і вирівняні по єдиній горизонтальній лінії.<sup>8</sup>

Така структура є еволюційним кроком порівняно зі звичайними суб-рутинами. При використанні суб-рутин у текстових мовах програміст бачить лише один рівень деталізації: він бачить виклик функції, але не бачить її вмісту. Силует дозволяє бачити **два рівні ієрархії одночасно**: макро-потік (перехід між гілками) та мікро-потік (алгоритм всередині кожної гілки) на одному екрані.<sup>3</sup>

## Правила виконання та побудови

Виконання Силуету концептуалізується як рух «бігуна» (runner) по графу<sup>3</sup>:

- Бігун стартує з початку найлівішої гілки.
- Досягнувши низу гілки, бігун натрапляє на ікону Address, яка містить ім'я наступної гілки.
- Бігун повертається до верхньої межі, ковзає вправо або вліво до гілки з відповідним ім'ям і починає спуск по її шампуру.
- Виконання завершується лише тоді, коли бігун досягає ікони End, яка згідно з правилами Силуету може розміщуватися виключно в кінці **крайньої правої гілки**.<sup>8</sup>

Фундаментальне правило побудови Силуету — гілки ніколи не повинні з'єднуватися між собою візуальними лініями напряму. Перехід здійснюється тільки через невидимий зв'язок між іконою Address та заголовком гілки.<sup>8</sup> Щодо просторового сортування, гілки повинні розташовуватись зліва направо у хронологічному порядку їх виконання. Стрибки назад (вліво) допускаються для реалізації ітеративних циклів (branch loop) або станів скінченного автомата, проте в чистих функціональних мовах (наприклад, Erlang) рух проти часу (назад до лівих гілок) жорстко обмежений.<sup>3</sup>

## 7. Readability & Best Practices

Досягнення стандарту «ясність понад усе» вимагає не просто механічного перетягування ікон, а стратегічного моделювання алгоритмічного потоку. Експертна розробка в DRAKON базується на кількох ключових принципах найкращих практик (best practices):

1. **Візуальні логічні формули (Visual Logic Formulas):** У традиційному програмуванні складні умови записуються в один рядок (наприклад, if (A and B or C)). DRAKON розглядає такий текстовий підхід як небажаний, оскільки він приховує проміжні стани оцінки.<sup>3</sup> Найкраща практика вимагає розбиття складної логічної формули на атомарні ікони If. Вони вибудовуються у візуальні патерни (у вигляді драбинки), які точно відображають еквівалентний порядок обчислення. Це робить явище "short-circuit evaluation" (ранній вихід з умови) явно видимим, рятуючи час під час зневадження.<sup>3</sup>
2. **Захист Спільної Долі (Common Fate):** Замість багаторазового дублювання одного й того ж блоку коду для обробки подібних помилок в різних місцях алгоритму, інженери повинні застосовувати маршрутизацію ліній до спільного блоку логіки на правій периферії діаграми.
3. **Мінімізація візуального шуму:** Хоча DRAKON автоматично забезпечує прямі кути, інженер несе відповідальність за те, щоб діаграма не перетворилась на горизонтальне простирadlo. Використання Силуету для розбиття довгих процесів є обов'язковим інструментом боротьби з візуальним шумом.<sup>9</sup>

## 8. Errors & Anti-patterns

Аналіз типових помилок (анти-патернів) під час конструювання DRAKON-схем вказує на ті ситуації, де інженери несвідомо повертаються до хаотичних звичок текстового програмування або класичних блок-схем.

- **Broken Skewer (Зламаний шампур):** Найбільш поширений анти-патерн, коли елементи, що складають «happy path», відхиляються від центральної вертикалі через неправильне трактування умов.<sup>3</sup> Це руйнує здатність читача просканувати алгоритм зверху вниз і створює "візуальний шум".
- **Нелегальна навігація в Силуэтах:** Спроба встановити посилання через ікону Address не на заголовок цільової гілки, а безпосередньо в середину її шампура.<sup>3</sup> Такий стрибок діє як класичний шкідливий оператор goto, створюючи неявні залежності та перетворюючи структурований граф на "Spaghetti code".
- **Порушення евристики успіху:** Проектування діаграми таким чином, що гілка обробки фатальної помилки залишається на центральному шампурі, а потік успішного продовження роботи виводиться праворуч. Це викликає когнітивний дисонанс, оскільки око експерта автоматично сприймає

відхилення вправо як негативну подію.<sup>3</sup>

- **Лівобічні виходи для умов:** Спроба заощадити місце на полотні шляхом виведення стрілки умови (No або Yes) вліво від ікони Question. Це грубе порушення топологічних правил мови, що руйнує геометрію Силуету і заборонено стандартами.<sup>3</sup>
- **Явні перетини ліній:** Виникають при спробі "спростити" зв'язки в редакторах, які не застосовують автоматичний манхеттенський роутинг. Перетин ліній є фундаментальним ворогом ясності й вимагає негайного рефакторингу логіки схеми шляхом введення додаткових гілок Силуету.<sup>3</sup>

## 9. Tools & Ecosystem

Екосистема розробки DRAKON пройшла еволюцію від ранніх закритих систем космічної індустрії (таких як GRAPHIT-FLOKS) до сучасних крос-платформних редакторів із відкритим вихідним кодом.<sup>2</sup>

**DRAKON Editor:** Де-факто стандартне середовище розробки на базі Tcl/Tk та SQLite.<sup>4</sup> Особливістю інструменту є його функція генератора коду. DRAKON Editor діє як компілятор, здатний транслювати візуальні діаграми у вихідний код широкого спектра мов, включаючи C, C++, C#, Java, JavaScript, Python, Tcl, Erlang, Lua, Go, D, AutoHotkey та Verilog.<sup>4</sup> Редактор підтримує інтеграцію з фреймворком Qt (генерація signals та slots у C++), забезпечує підтримку скінченних автоматів, і навіть має архітектуру для роботи з обміном повідомлень у моделі акторів (Actor model) в C#. <sup>4</sup> Крім того, він інтегрує мову **ERIL**, забезпечуючи можливість автоматичної генерації SQL-коду та структур даних з тих самих принципів візуалізації.<sup>4</sup>

**DrakonHub:** Сучасний веб-орієнтований редактор діаграм. На відміну від десктопного редактора, його бекенд працює на базі Tarantool 2, а сама система розроблена за допомогою DRAKON-JavaScript та DRAKON-Lua (їхні .js та .lua файли згенеровані безпосередньо з діаграм).<sup>17</sup> Це потужний інструмент колаборації, який, хоч і фокусується більше на документації бізнес-процесів та візуалізації, зберігає строгі правила мови.

**Drakon.Tech:** Професійна веб-орієнтована IDE, що спеціалізується на інтеграції з екосистемою JavaScript. Надає розширені можливості для онлайн-розробки та тестування логіки безпосередньо у браузері, дозволяючи обходити хаос глибоко вкладених колбеків у JS-програмуванні.<sup>1</sup>

**Fabula:** Офлайн-інструмент для створення діаграм з фокусом на початківців та навчання. Забезпечує можливості генерації псевдокоду з діаграм, що робить його корисним для освітнього середовища.<sup>1</sup>

Ці інструменти не лише підтримують малювання, але й забезпечують дотримання

ортогональної маршрутизації ліній і заборону перетинів, знімаючи з розробника необхідність ручного форматування.

## 10. Machine-readable Formats

Здатність візуальної мови легко інтегруватись із системами контролю версій та аналізаторами коду залежить від форматів її збереження. У контексті DRAGON існує два основних машиночитаних формати, кожен з яких слугує своїм архітектурним цілям.

**1. Формат файлу .drn (SQLite)** Основним форматом зберігання для десктопного DRAGON Editor є файл із розширенням .drn. З інженерної точки зору це не текстовий файл чи XML, а повноцінна реляційна база даних формату SQLite.<sup>5</sup> Внутрішня структура таблиць зберігає графічну репрезентацію ікон, текстовий вміст (код), взаємозв'язки між об'єктами та метадані для компіляції. Робота з файлами базується на стандартних SQL-транзакціях (через libsqlite3), що забезпечує атомарність оновлень діаграми, проте робить ручний мерджинг у системах типу Git складнішим порівняно з чистим текстом.<sup>5</sup>

**2. Текстовий формат JSON (DrakonWidget та DrakonHub)** Для забезпечення прозорості у веб-середовищі та можливості алгоритмічної генерації було розроблено чисту текстову JSON-репрезентацію графа (Diagram data model).<sup>6</sup> JSON-схема діаграми включає глобальні параметри (name, access, params, style) та ключовий об'єкт items. Вузол items діє як словник (хеш-таблиця), де ключем є унікальний строковий ідентифікатор ікони (наприклад, "12"), а значенням — об'єкт із полями:

- type: Тип ікони за специфікацією (action, question, branch, insertion тощо).
- content: Блок текстового коду або логіки (підтримує plain, markdown, html).
- one: ID наступної ікони в потоці, що створює топологічний зв'язок (directed edge).
- branchId: Вказівник на належність ікони до конкретної гілки макро-шаблону Силует.

Фундаментальна перевага цього формату полягає у **відсутності геометричних координат (X, Y)**. Формат зберігає виключно топологічне дерево виконання (AST). Детермінований рушій (DrakonWidget) автоматично парсить цей JSON і виконує рендеринг геометрії (розставляє вузли, малює шампури, розраховує ортогональні лінії), гарантуючи стовідсоткове дотримання ергономічних правил мови.

## 11. Engineering Implications for AI Generation

Відділення семантичної топології графа від його геометричного подання (X/Y позиціонування) робить архітектуру DRAGON ідеальною мішенню для систем штучного інтелекту. Великі мовні моделі (LLM), такі як GPT-4 чи Claude, за своєю

природою неефективні у розрахунках двовимірних просторових координат (генерування SVG-схем без перетинів зводиться до постійних галюцинацій).<sup>18</sup> Проте LLM демонструють видатні результати у генеруванні структур даних (JSON) та орієнтованих ациклічних графів (DAG).

Синергія AI та DRAKON працює за принципом **Гібридної генерації**:

- 1. LLM аналізує текстовий опис бізнес-процесу або неструктурований legacy-код.
- 2. Модель виконує логічну декомпозицію, розбиваючи процес на незалежні стани (генерує заголовки гілок для Силуету).
- 3. LLM визначає "happy path" та виділяє його у послідовність дій (мапінг у поле one вузлів). Експертні промпти наказують AI відводити винятки або обробники помилок у допоміжні гілки (реалізуючи правило "чим правіше, тим гірше").
- 4. LLM впливає лише топологічний JSON словник (формат DrakonWidget).
- 5. Детермінований рушій DRAKON рендерить цей JSON, забезпечуючи дотримання всіх Hard Constraints (заборона перетинів ліній, ідеальні кути, правильні відступи).

| Інженерне Завдання                          | Виконавець (LLM / Deterministic / Hybrid) | Рівень Складності | Технічні Примітки                                                                                    |
|---------------------------------------------|-------------------------------------------|-------------------|------------------------------------------------------------------------------------------------------|
| Екстракція логіки та декомпозиція алгоритму | LLM                                       | Низька            | LLM чудово структурує неструктурований текст на логічні кроки та гілки для об'єктів типу Silhouette. |
| Визначення "Happy Path"                     | LLM                                       | Низька            | Навчається через System Prompt: "Спрямуй найбільш вірогідний сценарій по центральній осі вузлів".    |

|                                                  |                      |         |                                                                                                                                           |
|--------------------------------------------------|----------------------|---------|-------------------------------------------------------------------------------------------------------------------------------------------|
| Топологічне зв'язування вузлів (Генерація one)   | LLM                  | Середня | Модель повинна слідкувати за коректністю ID вказівників, особливо при формуванні зворотних петель для ікон For loop або Address.          |
| Розрахунок XY координат та ортогональний роутинг | Deterministic Engine | Висока  | Геометрична маршрутизація ліній без перетинів повністю делегується зовнішньому алгоритму рендерингу (DrakonWidget). AI цим не займається. |
| Перевірка валідності графа (одна точка End)      | Hybrid               | Низька  | LLM генерує структуру, але валідатор JSON схеми може відхилити запит, якщо знайде порушення топологічних інваріантів.                     |
| Оцінка ергономіки "Чим правіше, тим гірше"       | LLM (Critic Agent)   | Середня | Оскільки рушій не розуміє семантики "помилки", спеціальний AI-агент повинен верифікувати, чи дійсно критичні збої відправлені в праву     |

|  |  |  |                  |
|--|--|--|------------------|
|  |  |  | частину Силуету. |
|--|--|--|------------------|

## 12. Open Questions & Missing Materials

Попри детальний аналіз екосистеми, в наявному корпусі джерел залишаються білі плями, які вимагають додаткового глибокого дослідження (Deep Research).

● **Вичерпний перелік графічного алфавіту:** Джерела багаторазово підтверджують статистику про 27 базових ікон та 21 макроікону.<sup>2</sup> Однак, у документації перераховано семантику лише близько 20 основних елементів. Невідомі точні назви та архітектурне призначення залишкових макроікон. ●

**Детальна DDL-схема SQLite файлів:** Відомо, що основний формат .drn є реляційною БД SQLite.<sup>5</sup> Однак відсутня специфікація Data Definition Language (DDL) — точна структура таблиць, первинних ключів та зв'язків. Відсутність цієї схеми ускладнює розробку кастомних конвертерів або інструментів для безконфліктного злиття (merge) у системах CI/CD без застосування реверс-інжинірингу. ●

**Імплементация систем реального часу:** Джерела вказують на підтримку DRAKON-2 операторів реального часу, таких як таймери та управління конкурентністю.<sup>3</sup> Однак залишається незрозумілим, як візуальні ікони Control period або Process фізично транскрибуються в низькорівневий код (наприклад, у нативні треди C/C++ чи асинхронні проміси в JavaScript).

## 13. Curated Bibliography

Для подальшого вивчення або інтеграції алгоритмів генерації DRAKON рекомендується звернутись до наступного корпусу первинних та вторинних джерел:

1. *Офіційний репозиторій DRAKON Editor*. Міткін, С. Документація з підтримки мов програмування, ERIL, кінцевих автоматів та інтеграції з SQLite..<sup>4</sup>
2. *Специфікація DrakonWidget JSON*. Опис структури об'єктів для текстової репрезентації діаграм (items, branchId, type) та відстеження змін (pushEdit).<sup>6</sup>
3. *Parondzhanov, V., "A Graphic Language DRAKON" (PDF)*. Фундаментальний мануал з обґрунтуванням принципів "Happy path", когнітивної ергономіки та заборон на перетин ліній.<sup>3</sup>
4. *DRAKON Reference Guide (DrakonFlow)*. Найповніша доступна документація з семантики іконографії (Shelf, Control period, Timer, Process).<sup>14</sup>
5. *Erlang Silhouette Manual*. Архітектурні правила побудови багатогілкових діаграм типу Силует, трансляція в кінцеві автомати, заборона горизонтальних з'єднань між гілками.<sup>8</sup>
6. *ERIL: Візуальна мова моделювання даних*. Інструкція Степана Міткіна з

перенесення Манхеттенської геометрії DRAKON на дизайн реляційних зв'язків (один-до-багатьох, володіння, агрегація).<sup>11</sup>

---

## Executive Summary

Візуальна мова DRAKON, зароджена в надрах програми космічних човників «Буран», є унікальним інструментом інженерного моделювання, що радикально знижує когнітивне навантаження під час аналізу складних алгоритмів. Поєднуючи абсолютну математичну строгість із жорстко контрольованою геометричною ергономікою (відсутність перетинів, ортогональні лінії, спрямування потоку «зверху вниз»), мова забезпечує безпрецедентну ясність («happy path» завжди лежить на центральній осі). Завдяки декомпозиційним конструкціям, таким як Силует, та можливості генерації коду безпосередньо у формальні мови програмування, DRAKON виходить за межі звичайних блок-схем. Топологічна структура діаграм, відв'язана від координат шляхом представлення у JSON-форматах, робить DRAKON ідеальним кандидатом для автоматизованої генерації архітектури програмного забезпечення за допомогою сучасних LLM-систем штучного інтелекту.

## Structured Findings

- **Когнітивна оптимізація:** DRAKON застосовує концепцію «Манхеттенської маршрутизації» та гравітаційного читання, повністю усуваючи візуальний шум (перетин ліній або діагоналі), що притаманний діаграмам UML та звичайним блок-схемам.
- **Правило головного маршруту:** Найбільш успішний сценарій виконання завжди формує прямий вертикальний «шампур». Будь-які помилки чи винятки просторово відхиляються вправо за принципом «чим правіше, тим гірше».
- **Декомпозиція через Силует:** Складні алгоритми діляться на паралельні гілки, що відображаються на одному екрані, дозволяючи читачу охопити загальну логіку (макрорівень) та деталі обчислень (мікрорівень) без переходу до вкладених функцій.
- **Універсальність гібридної мови:** Візуальні ікони відповідають за логічний потік, тоді як їхній внутрішній контент може містити нативний код (C++, Java, JavaScript), з якого десктопний DRAKON Editor або DrakonHub генерує готові скрипти.
- **Синергія з AI:** JSON-схема DrakonWidget описує графи зв'язків без зазначення піксельних X/Y координат. Це ідеально підходить для LLM, які можуть генерувати топологічну логіку, делегуючи складне геометричне розташування детермінованим алгоритмам.
- **Моделювання структур даних:** Споріднена мова ERIL адаптує жорсткі візуальні правила DRAKON для проєктування реляційних баз даних та

об'єктного успадкування.

## Unresolved Questions

- Де знайти вичерпну академічну специфікацію повного алфавіту DRAKON, що деталізує семантику усіх 27 ікон та 21 макроікони?
- Якою є точна реляційна структура DDL (таблиці, ключі) внутрішньої бази даних SQLite, яка використовується у файлах розширення .drn?
- Який механізм та синтаксис компіляції використовуються для трансляції візуальних ікон паралелізму (Concurrent processes, Process, Timer) у нативні потоки цільових мов програмування в архітектурі DRAKON-2?

## Recommended Next Research Tasks

- **Тема для Промпту 2 (AI Generation Architecture):** Розробка еталонного набору System Prompts для LLM-агентів, здатних екстрагувати неструктуровану текстову бізнес-логіку та трансформувати її у строгий топологічний JSON формату DrakonWidget, враховуючи просторове рознесення виняткових ситуацій згідно з правилом «чим правіше, тим гірше».
- **Тема для Промпту 3 (Database Reverse-Engineering):** Глибокий аналіз архітектури файлів .drn. Здійснення реверс-інжинірингу реляційної структури таблиць SQLite на основі вихідних кодів парсера Tcl, для створення кастомних CLI-інструментів (merge, diff) у середовищі CI/CD.
- **Тема для Промпту 4 (ERIL Data Modeling):** Детальне дослідження мови ERIL як аналога DRAKON для структур даних: мапінг візуальних концепцій (vertical ownership, horizontal peer links) до генерації абстрактних схем SQL та NoSQL баз даних.

## Джерела

1. DRAKON visual programming language — communication tool for ..., доступ отримано травня 9, 2026, <https://www.mql5.com/en/articles/13324>
2. DRAKON - Wikipedia, доступ отримано травня 9, 2026, <https://en.wikipedia.org/wiki/DRAKON>
3. DRAKON.pdf, доступ отримано травня 9, 2026, <https://drakon-editor.sourceforge.net/DRAKON.pdf>
4. drakon\_editor/README.md at master · stepan-mitkin/drakon\_editor ..., доступ отримано травня 9, 2026, [https://github.com/stepan-mitkin/drakon\\_editor/blob/master/README.md](https://github.com/stepan-mitkin/drakon_editor/blob/master/README.md)
5. stepan-mitkin/drakon\_editor: DRAKON Editor · GitHub - GitHub, доступ отримано травня 9, 2026, [https://github.com/stepan-mitkin/drakon\\_editor](https://github.com/stepan-mitkin/drakon_editor)
6. stepan-mitkin/drakonwidget: A JavaScript widget for ... - GitHub, доступ отримано травня 9, 2026, <https://github.com/stepan-mitkin/drakonwidget>

7. DRAKON Visual Language: Tutorial. Part 1 | PDF - Slideshare, доступ отримано травня 9, 2026, <https://www.slideshare.net/slideshow/drakon-part1-eng/22563246>
8. DRAKON-Erlang part 6 - DRAKON Editor, доступ отримано травня 9, 2026, <https://drakon-editor.sourceforge.net/drakon-erlang/silh.html>
9. Our Accelerators | Drakon Mapping - VMG Labs, доступ отримано травня 9, 2026, <https://vmglabs.com/drakon-mapping>
10. Файл:Macroicons of Visual Programming Language DRAKON.png - Википедия, доступ отримано травня 9, 2026, [https://ru.wikipedia.org/wiki/%D0%A4%D0%B0%D0%B9%D0%BB:Macroicons\\_of\\_Visual\\_Programming\\_Language\\_DRAKON.png](https://ru.wikipedia.org/wiki/%D0%A4%D0%B0%D0%B9%D0%BB:Macroicons_of_Visual_Programming_Language_DRAKON.png)
11. ERIL - Wikipedia, доступ отримано травня 9, 2026, <https://en.wikipedia.org/wiki/ERIL>
12. DRAKON - HandWiki, доступ отримано травня 9, 2026, <https://handwiki.org/wiki/DRAKON>
13. The ERIL Language - DRAKON Editor, доступ отримано травня 9, 2026, <https://drakon-editor.sourceforge.net/eril.html>
14. DRAKON Language Reference|DrakonFlow, доступ отримано травня 9, 2026, <https://drakonflow.com/read/drakon-reference>
15. Solving Problems with Hard and Soft Constraints Using a Stochastic Algorithm for MAX-SAT, доступ отримано травня 9, 2026, <https://www.cs.cornell.edu/selman/papers/pdf/maxsat.pdf>
16. How to Classify Hard and Soft Constraints in Non-binary Constraint Satisfaction Problems - UPV, доступ отримано травня 9, 2026, <https://personales.upv.es/misagre/papers/ai2003.pdf>
17. GitHub - stepan-mitkin/drakonhub: The source code for DrakonHub ..., доступ отримано травня 9, 2026, <https://github.com/stepan-mitkin/drakonhub>
18. Turn your data into diagrams you can read | ToDiagram, доступ отримано травня 9, 2026, <https://todigram.com/>